Practical Work: DevOps Infrastructure & Automation with Ansible & Terraform

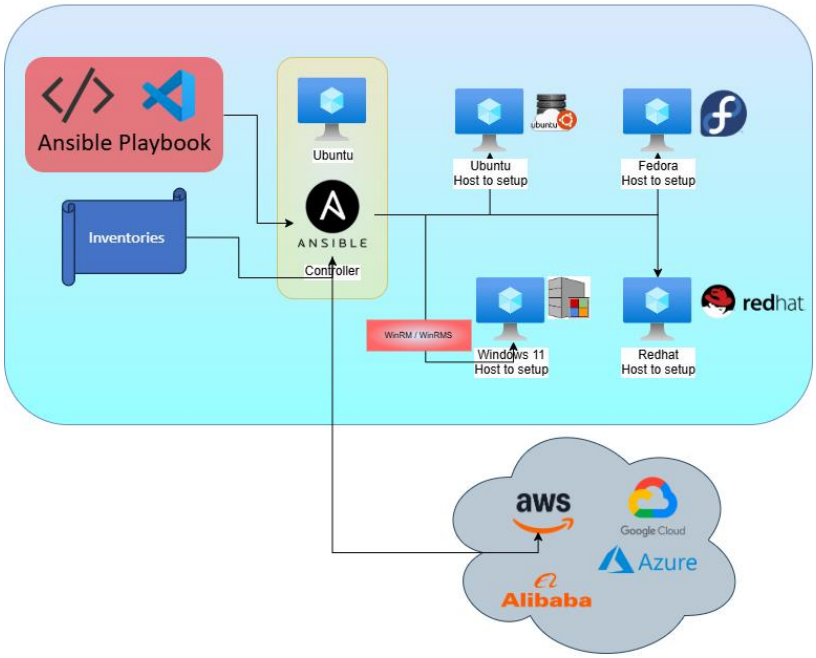
Professor/Author: Issiaka KONE

School: EFREI

Lecture: DevOps - Culture, Practices and Tools (CI/CD, IaC, Observability, Security)

Classroom: M1-DEV1 (2026)

Target Date: 25-05-2026



1. Objective and Architecture Overview

The goal of this practical work is to build, provision, and automate a multi-OS infrastructure using Gitlab CI/CD, Terraform, and Ansible. You will set up a local hybrid environment using Virtual Machines (VMs) that simulates an enterprise automation controller managing various target nodes (Linux and Windows), as well as interfacing with public cloud providers.

Infrastructure Blueprint

Based on the **Ansible Tutorial Architecture 2026** design slide of the course, your infrastructure must include:

- **Ansible Controller:** An Ubuntu-based machine running Ansible, acting as the core execution engine.
- **Target Hosts (Managed Nodes):**
    - Ubuntu Linux Node
    - Fedora Linux Node

    - RedHat Linux Node
    - Windows 11 Node (managed via **WinRM**/**WinRMS**)
- **Cloud Integrations:** The control plane must be ready to interface with AWS, GCP, Azure, and Alibaba Cloud.

2. Detailed Roadmap & Technical Requirements

Step 1: Hypervisor and Base OS Installation

1. Download and install **VirtualBox**.
2. Download the latest stable **Ubuntu Server/Desktop** ISO.
3. Install the Ubuntu VM. This VM will serve as your primary environment hosting your automation tools and CI/CD agent.

Step 2: Custom GitLab Runner Setup

1. Install the **GitLab Runner** binary on your Ubuntu VM following the Official GitLab Runner Documentation.
2. Register and connect this custom runner to your private GitLab repository/account.
3. Configure your repository's CI/CD settings to ensure that any push event to the main branch automatically triggers a pipeline executing *only* on your custom local runner.

Step 3: Infrastructure-as-Code with Terraform

1. Install **Terraform** inside your execution environment.
2. Within your GitLab CI/CD pipeline, integrate a Terraform step.
3. Use a **Terraform Jinja2 template file (.tpl)** to dynamically generate your Ansible inventory file, mapping out the IP addresses and connection variables of your target hosts.

Step 4: Configuration Management with Ansible

1. Install and configure **Ansible** on the Controller.
2. Create a playbook to achieve the following automation sequence through the GitLab pipeline:
    - **Local Analysis:** Gather facts on the localhost (Ansible Controller).
    - **Linux Timezone Management:** Programmatically update the system **timezone** of your target Linux nodes to **Europe/Paris**, verify the change, and then switch it to **Africa/Abidjan** (GMT).

- **Windows Node Management:** Boot a Windows 11 Virtual Machine, configure **WinRM / WinRMS** for remote administration, and execute the exact same timezone modification workflow (Europe/Paris then Africa/Abidjan) via Ansible.

3. Evaluation and Delivery Instructions

Expected Deliverables

You must submit a single compressed **.zip** archive containing:

1. **Your GIT Repository:** The complete source code folder (including your **.gitlab-ci.yml**, Terraform files, Jinja2 templates, Ansible playbooks, and roles).
2. **Documentation (README.md** and/or **Word/PDF file):** A detailed Markdown file at the root of your repository explaining:
    - The overall architecture and how the components interact.
    - Step-by-step instructions on how you configured the infrastructure.
    - How to run and test the setup.
3. **Pipeline Proof (Screenshots):** * A clean, full-screen screenshot of your successfully completed **GitLab pipeline log**.
    - **Crucial:** The screenshot must clearly highlight and prove that the pipeline was executed by your **custom local GitLab Runner**, not a shared GitLab-hosted runner.

Formatting Rules & Deadline

- **File Naming Convention:** The zip file **MUST** be named exactly as follows (replace with your actual name):

  → LASTNAME Firstname – DevOps M1-DEV1 2026 – 09-04-2026.zip

- **Strict Penalty:** Any variation in the filename syntax, missing logs, or late submissions will result in a direct grading penalty.

4. Grading Criteria (Scale: /20)

Criteria	Description	Points
VM & Runner Setup	**VirtualBox** operational, GitLab Runner correctly registered and acting as the exclusive pipeline executor	4 pts
Terraform & Jinja2	Dynamic inventory generation from templates inside the pipeline stage	4 pts

Ansible Automation	Fact gathering, cross-platform **timezone** management (**Linux** & **Windows** via **WinRM**)	6 pts
Architecture Alignment	Respecting the provided multi-node diagram layout and cloud-ready specifications	3 pts
Delivery & Rigor	Accurate naming convention, comprehensive README.md, clear screenshot proofs	3 pts

Tutorials: Introduction to DevOps: Step by Step

1. Install VirtualBox
2. Download Ubuntu
3. Install Ubuntu
4. Install Gitlab Runner
   - https://docs.gitlab.com/runner/install
5. Setup and connect the runner to your Gitlab account
6. Use your repo to start a pipeline using your custom runner on push to the main branch
7. Install Terraform and within your pipeline create the inventory file from Terraform Jinja 2 template file
8. Setup Ansible
9. Use your pipeline to execute Ansible to
   - Gather facts on the localhost
   - Setup the time zone to **Paris** and then to **Africa/GMT**
10. Do the same to a new Windows virtual machine with **WinRM** connection
11. Delivery
    1. Your GIT repo with a readme explaining all the steps and architectures, etc.
    2. The full screenshot of your Gitlab pipeline log with special emphasis on the used runner
    3. All in a single zip file with mandatory filename as: «**LASTNAME Firstname – DevOps M1-DEV2 2026 – 09-04-2026.zip**»